EXITS DB

2º DAM IES BARAJAS 2018

Asignatura: Acceso a Datos

Alumno: José Luis Carro Sesmero

ÍNDICE

- 1. Introducción
- 2. Características
- 3. Instalación
- 4. Consultar datos
- 5. Insertar, modificar, borrar, alterar y renombrar nodos

1. Introducción

- Es un sistema de base de datos orientado a documentos NoSQL.
- eXist proporciona Xquery y XSLT como su consulta y lenguajes de programación de aplicaciones.
- Se lanzó bajo la versión 2.1 de GNU LGPL.
- Creado en 2000 por Wolfgang Meier.
- eXist fue galardonado con la mejor base de datos XML del año por InfoWorld en 2006.

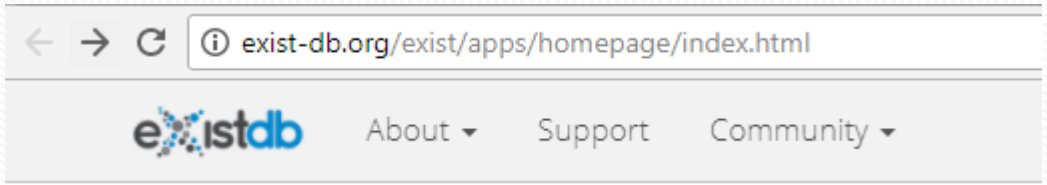
2. Características

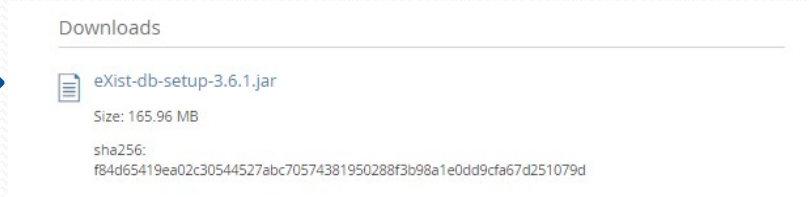
- Admite interfaces REST para interactuar con formularios web de tipo AJAX .
- Permite a los desarrolladores de software persistir documentos XML / JSON / Binary sin escribir extensos middleware.
- Las aplicaciones como XForms pueden guardar sus datos usando solo unas pocas líneas de código.
- La interfaz WebDAV para eXist permite a los usuarios "arrastrar y soltar" archivos XML directamente en la base de datos eXist.

3. Instalación

- <u>Paso 1:</u>

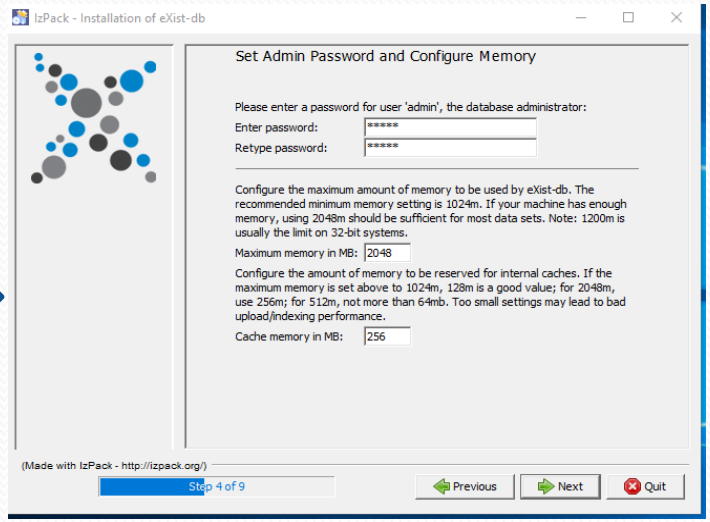
  Descargarnos Exits Db en la página oficial como muestro en la imagen:





3. Instalación

Paso 2: Al hacer doble clic derecho nos aparecerá la ventana principal de exits-db para realizar la instalación que se realizará en 9 pasos:



3. Instalación

- <u>Paso 3:</u>
- Ahora hay que iniciar el servicio

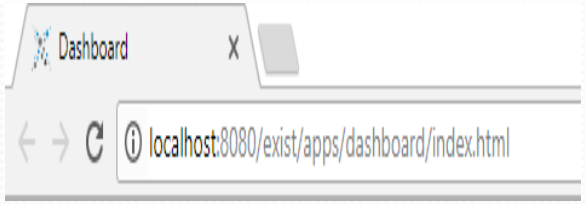
```
C:\eXist-db>java -jar start.jar

C:\eXist-db>
```

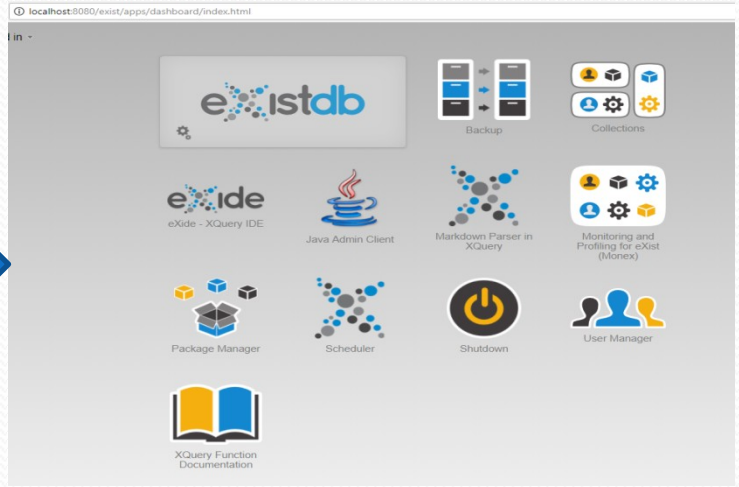
- Para pararlo usaremos en C:\eXist-db\bin la siguiente línea de comandos:
- Shutdown.bat –p admin(contraseña de admin).

- Recordar que el puerto por defecto es 8080 y sino 8443.

3. Instalación

- Por último navegamos a la ruta localhost:8080 para empezar a utilizar eXits-Db y nos aparecerá la ventana principal (DashBoard):





5. Consultar datos

- Las consultas se realizan mediante expresiones FLOWER:
- **FOR:** asocia una o más variables a expresiones, creando un conjunto de tuplas en el cual cada tupla vincula una variable dada a uno de los ítems a los cuales está asociada la expresión evaluada.
- **LET:** vincula las variables al resultado de una expresión, agregando estos vínculos a las tuplas generadas por la cláusula FOR.
- **WHERE:** filtra tuplas, quedando solo aquellas que cumplen con la condición.
- **ORDER BY:** ordena las tuplas en el conjunto de tuplas.
- **RETURN:** construye el resultado de la expresión FLWOR para una tupla dada.

- Este ejemplo nos saca el dni de las personas que tienen más de 18 años.

```
 for $x in doc("agenda.xml")/agenda/persona
where $x/edad>22
order by $x/edad
return $x/dni
```

- Este ejemplo nos saca los teléfonos de las personas que tienen menos de 22 años.

```
for $x in doc("agenda.xml")/agenda/persona
  where $x/edad<22
  order by $x/edad
  return $x/telefono
```

- Este ejemplo nos saca los nombres de las personas entre etiquetas personalizadas llamadas nombres.

```
for $x in doc("agenda.xml")/agenda/persona/nombre
return
<nombres>{$x} </nombres>
```

- Este ejemplo nos saca los nombres de las personas entre etiquetas personalizadas llamadas nombres pero todos de una sola vez por lo que es más cómodo.

```
let $x:=doc("agenda.xml")/agenda/persona/nombre
return
<nombres>{$x}</nombres>
```

- En este ejemplo nos saca los datos de una persona introduciendo su apellido2.

  doc(“agenda.xml”)/agenda/persona[substring(apellido2, 1, 5)=“Pérez”]

Insertar nodos

- update insert <email type="gmail">laura@gmail.com</email> into //agenda/persona[1]

Modificar nodos

- update replace /agenda/persona[1]/nombre with <nombre>Rosa</nombre>

Borrar nodos

- for $dni in //agenda/persona/dni return update delete $dni

Alterar valores nodos

- Update value /agenda/persona[1]/nombre[1] with 'Laura'

Renombrar nodos

- for $x in /agenda/persona/dni return update rename $x as 'años'